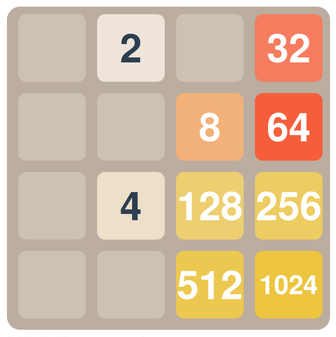
Solving 2048: Classical RL vs. LLM-Guided RLVR

Shriman Raghav[1] Gautham Ramkumar[1]

[1]MS Robotics, Khoury College of Computer Sciences, Northeastern University

CS 5180: Reinforcement Learning | Prof. Robert Platt

Problem: The 2048 Puzzle



Slide tiles in 4 directions. Equal tiles merge and double. Reach **2048** to win.

Challenges:

- $\sim 10^{19}$ reachable board states
- Sparse, delayed reward signal
- Stochastic tile spawns (2 or 4)
- 100–1,000+ steps per episode
- Up to 4 invalid actions per state

MDP Formulation:

$\mathcal{S}$ 4×4 board → 16-ch binary tensor $\{0,1\}^{16\times4\times4}$

$\mathcal{A}$ {UP, RIGHT, DOWN, LEFT}; invalid masked to $-\infty$

$\mathcal{R}$ Δscore + milestones: $+50$@256, $+100$@512, $+200$@1024

γ 0.99 (long-horizon credit assignment)

Design of Experiments

Research Question: Can LLM-guided RLVR (GRPO: Group Relative Policy Optimization) match classical RL agents on a structured combinatorial game under equivalent compute constraints?

Agent Lineup: 6 Classical + 2 LLM:

Agent	Category	Impl.
DQN	Off-policy, value-based	Custom PyTorch
SAC (Discrete)	Off-policy, max-entropy	Custom PyTorch
QR-DQN	Off-policy, distributional	SB3
PPO	On-policy, clipped	SB3
A2C	On-policy, actor-critic	SB3
Semi-grad SARSA	On-policy, linear FA	Custom NumPy
GRPO 0.5B	LLM-RLVR	trl
GRPO 1.5B	LLM-RLVR	trl

Shared Experimental Controls:

- All deep agents: **identical CNN backbone** (3-layer, 330K params)
- Same shaped reward function and 2048 gym environment
- Invalid action masking applied uniformly across all agents
- Evaluation: last 100 episodes + unlimited "Hunt" (best-of-N) mode

Training Protocol:

Classical (1M ckpt):	1M env steps	intermediate eval
Classical (5M ckpt):	5M env steps	≈2 hrs, RTX 4060
GRPO:	300 prompts × 4	≈12 hrs, RTX 4060

CNN Backbone (shared): Conv(16, 3×3) → Conv(32, 3×3) → Conv(64, 3×3) → FC(512) → Q(4). Input: 16-channel binary encoding of tile values. **330K params**. Score differences reflect the *learning algorithm only*.

Algorithms

DQN ★ Best Performer

Custom PyTorch

Off-policy, value-based

Buffer: 100K transitions
Batch: 64
ε: $1.0 \to 0.01$ / 100K
Target: hard sync / 1K
LR: 10^{-4} (Adam)
Grad clip: 1.0

```
DQN Update:
1. Sample (s,a,r,s') from B
2. Mask invalid a' to −∞
3. y ← r + γ max_a' Q_θ̄(s',a')
4. L ← SmoothL1(Q_θ, y)
5. θ ← θ − α∇L
6. Sync θ̄ ← θ every C steps
```

Why DQN wins: Replay buffer revisits high-tile transitions thousands of times. States with tile ≥ 512 (<0.1% of all steps) are repeatedly replayed. On-policy methods see each transition **exactly once**: no second chance.

SAC (Discrete)

Custom PyTorch

Off-policy, max-entropy

$J(\pi) = \sum_t \mathbb{E}[r_t + \alpha\mathcal{H}[\pi(\cdot|s_t)]]$

Buffer: 200K transitions
Batch: 256
Polyak τ: 0.005
LR: 10^{-4} (Adam)
Warmup: 10K steps
Target ent.: $0.4 \log|\mathcal{A}|$

```
SAC Update:
1. Sample (s,a,r,s') from B
2. Mask invalid actions in π
3. y ← r + γ(Q_min − α log π)
4. Update twin Q_1, Q_2 critics
5. Update π via entropy objective
6. Auto-tune α to target entropy
```

Max-entropy bonus diversifies tile placement but over-explores in sparse-reward episodes. Soft Polyak update (τ=0.005) slows critic convergence vs. DQN's hard sync. Avg score: **1,255**, $6\times$ below DQN.

PPO (MaskablePPO)

Stable-Baselines3

On-policy, clipped actor-critic

Rollout: 2048 steps × 8 envs
Batch: 64
Epochs: 10 per rollout
LR: 3×10^{-4}
GAE: λ=0.95
Clip: ϵ=0.2
Entropy: c_e=0.01

```
PPO Update:
1. Collect rollout (T steps, 8 envs)
2. Compute GAE adv. Â
3. r_t(θ) = π_θ/π_old
4. L = min(r_t Â, clip(r_t) Â)
5. Update θ for K=10 epochs
6. Discard all rollout data
```

Why PPO fails: Each $\sim$400-step episode used **once** then discarded. Effective discount $\gamma^{400}\approx0.018$ kills gradient signal for early moves. Score **drops 12%** from 1M→5M steps: active policy degradation.

A2C

Stable-Baselines3

On-policy, actor-critic

No clipping; single gradient step.

n_steps: 5
LR: 7×10^{-4}
c_e: 0.01
c_v: 0.5

$\mathcal{L} = -\mathbb{E}[\log\pi\hat{A}] \quad +0.5\|R-V\|^2 - 0.01\mathcal{H}$

```
A2C Update:
1. Collect 5-step rollout
2. Â ← R − V(s)
3. L_π ← −log π Â
4. L_v ← 0.5‖R−V‖²
5. L ← L_π + L_v − c_e H
6. Single gradient step on L
```

5-step bootstrap limits credit in 400-step episodes. Yet **beats PPO by +982 avg**: faster LR (7×10^{-4}) and synchronous updates converge better early on. Simpler objective = more stable gradient.

QR-DQN (Distributional)

Stable-Baselines3

Off-policy, distributional

Learns full return distribution via N=50 quantiles rather than $\mathbb{E}[Q]$.

Buffer: 100K
N: 50 quantiles
Output: 4×50=200 values
LR: 10^{-4}
Target: sync / 1K steps

```
QR-DQN Update:
1. Sample (s,a,r,s') from B
2. For each quantile τ_i:
δ_ij = r + γθ_j(s') − θ_i(s)
3. L = Σ_i ρ^κ_τi(δ_ij)
4. a* = arg max_a (1/N) Σ_i θ_i
```

Action selection by mean(Q) is **identical to DQN**. Distributional modeling provides no behavioral gain. Quantile regression on non-stationary 2048 targets causes instability: score **degrades** -9% from 1M→5M.

Semi-gradient SARSA (S&B Alg. 10.1)

Custom NumPy

On-policy, linear FA (no neural net!)

$\hat{q}(s,a,\mathbf{w}) = \mathbf{w}_a^\top\phi(s)$

$\mathbf{w} \in \mathbb{R}^{4\times30}$ = **120 params**

α: 10^{-3}
ε decay: 0.99995/step
$\varepsilon_{\min}$: 0.05

30-dim $\phi(s)$ (S&B §9.5):

- $\log_2$(tile) values $\times16$
- Empty cell ratio, merge potential
- Monotonicity L/R/U/D ($\times4$)
- Corner bonus, smoothness ($\times2$)
- Snake pattern, edge sum ($\times2$)
- Bias/interaction terms ($\times4$)

```
SARSA Update (Alg 10.1):
1. Observe s, choose a (ε-greedy)
2. Take a, observe r, s'
3. Choose a' (ε-greedy on s')
4. δ ← r + γq̂(s',a') − q̂(s,a)
5. w_a ← w_a + αδφ(s)
6. s ← s', a ← a'
```

120 params beats 330K+! Game-specific inductive bias (monotonicity, snake) outperforms CNN on on-policy data. Validates S&B §9.5.

GRPO: Group Relative Policy Optimization

trl + transformers

GRPO (Group Relative Policy Optimization, DeepSeek-R1, 2025): fine-tunes **Qwen2.5-0.5B / 1.5B** via verifiable rewards *without* a critic. Board → ASCII text → chain-of-thought reasoning → action token. No replay buffer.

Group-normalized advantage:

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G + \epsilon}, \quad G=4$$

3-stage reward scheduling:

1. Format compliance (XML tags)
2. Game reward (Δscore)
3. Thinking quality (token bonus)

Compute cost: 4 LLM forward passes per training step. 300 prompts ≈ **12 hrs** on RTX 4060 vs. **2 hrs** for 5M classical steps.

```
GRPO Update:
1. Sample G=4 completions
2. Score each: r_i ← verifier
3. Â_i = (r_i − μ_G)/σ_G
4. L = −Σ_i Â_i log π_θ
−β KL(π_θ‖π_ref)
5. Update θ; repeat for next prompt
```

LLM sees 1,200 states vs. DQN's 5M. Produces **interpretable CoT** per move. $3\times$ model → $3.3\times$ score: **scaling is the lever**.

```
Input (Qwen2.5-1.5B, move 41):
Board: [[0,0,4,8],[0,4,16,32],
[0,0,64,128],[0,0,0,256]]
Max tile: 256 | Score: 956
<think>
256 anchored bottom-right - good.
Gradient: 128, 64, 32, 16 intact
along right/bottom edges.
RIGHT merges 4+4 in row 1 (+8).
Avoids scattering corner tile.
</think>
<answer>RIGHT</answer> ✓ +8
```

Spatial Failure (0.5B):

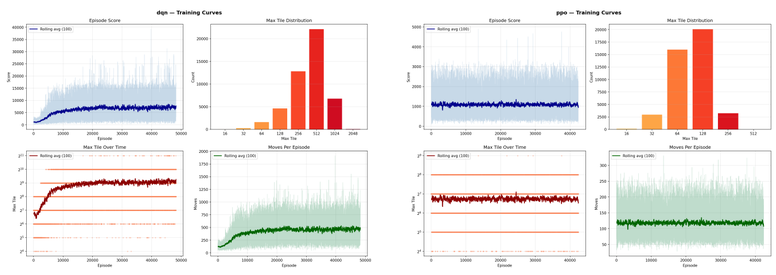
Board: `[[0,4,0,0],[4,0,8,0],...]`

Model: *"4 at [0][1] and [1][0] are adjacent – LEFT merges them into 8"*

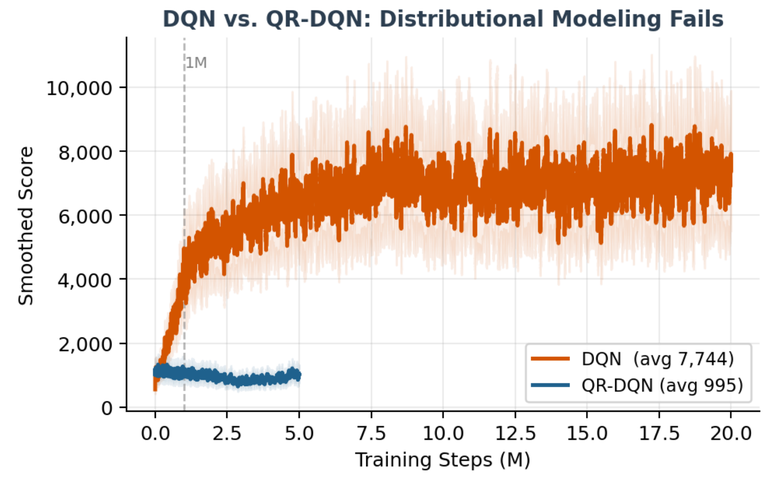
Reality: they are in different rows. LEFT moves each to col 0 independently. No merge occurs.

Root cause: LLM pattern-matches on **token proximity**, not grid geometry. Cannot mentally simulate a slide.

Training Curves (5M Steps)



DQN: avg 7,744 ★ PPO: avg 962 ↓ (degrades!)

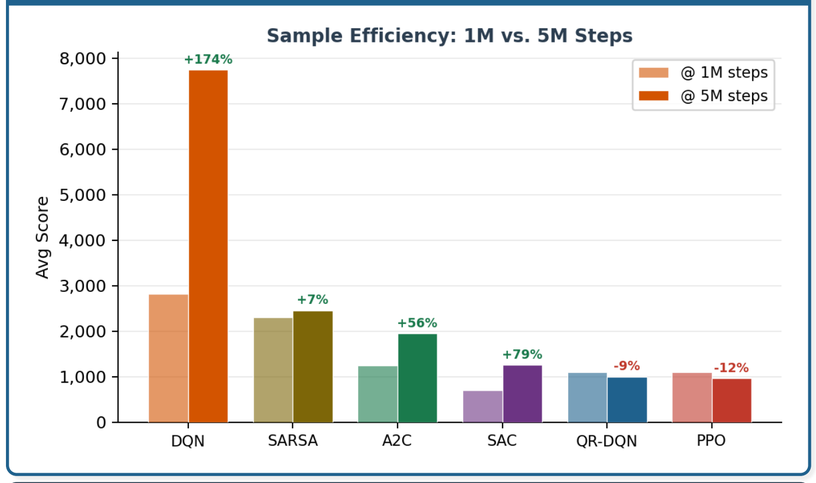


SARSA: 2,456 | SAC: 1,255 | A2C: 1,944 (see table below)

Final Performance (5M Steps)

Agent	Avg Score	Max Score	Max Tile	512+ Rate	Params	Impl.
DQN ★	7,744	39,314	2048	76%	330K	Custom
SARSA	2,456	11,436	1024	6%	120	NumPy
GRPO 1.5B	2,348	2,348	256	0%	1.8B	trl
A2C	1,944	9,640	1024	6%	330K	SB3
GRPO 0.5B	1,816	1,816	128	0%	494M	trl
SAC	1,255	7,882	512	<1%	330K	Custom
QR-DQN	995	5,176	512	<1%	430K	SB3
PPO	962	4,900	512	<1%	330K	SB3

Sample Efficiency: 1M → 5M Steps



Hunt Mode: Can Any Agent Reach 2048?

Unlimited attempts with ε=0.05 exploration. Runs until tile 2048 reached or 3K attempts.

Agent	Attempts	Best Tile	Best Score	Verdict
DQN ★	3,902	2048	21,572	Only success
SARSA	3,000	1024	12,300	Closest (120 params!)
A2C	3,000	512	7,844	Stalls post-512
SAC	3,000	256	3,648	Entropy hurts late
PPO	193	64	520	Collapses early

DQN is the only agent to reach 2048 in 3,902 unlimited attempts (0.026% rate). Replay buffer is the decisive factor even in best-case conditions. SARSA reaches 1024 with 120 params; PPO degrades to tile 64 under hunt pressure.

Classical RL vs. RLVR

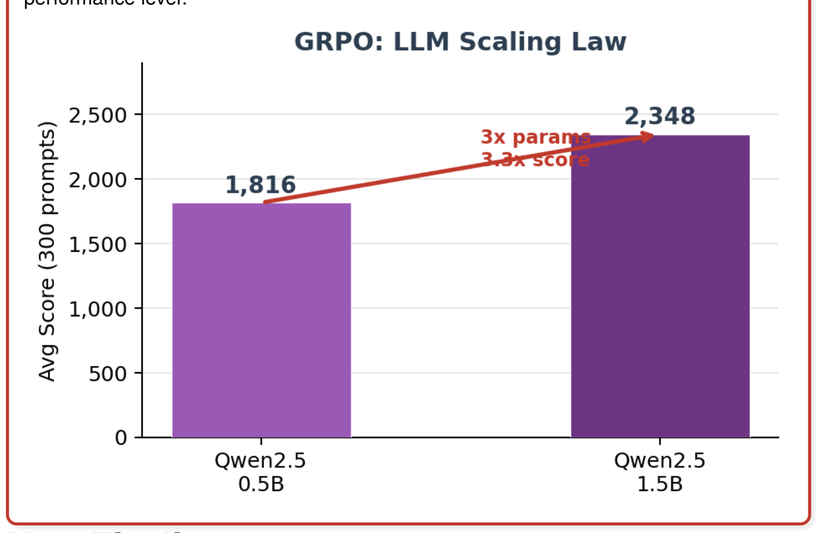
Head-to-Head Comparison

Dimension	Classical RL	LLM-RLVR (GRPO)
Avg Score	7,744 (DQN)	2,348 (1.5B)
Max Tile	2048 (DQN)	256 (1.5B)
Parameters	120 – 430K	494M – 1.8B
Inference	<4 ms/move	1,500 ms/move
Training data	5M env steps	300 prompts
Interpretable	No (black box)	Yes: chain-of-thought
Observation	Tensor $16\times4\times4$	ASCII board text
Action space	Integer $\{0,1,2,3\}$	Natural language token

Score gap: DQN replays 5M transitions; GRPO sees 1,200 board states. Tensor input is dense information; ASCII forces tokenization overhead.

RLVR strength: Each GRPO move is paired with an inspectable reasoning trace. Classical agents are complete black boxes with no explainability.

Scaling law: 0.5B → 1.5B ($3\times$ params) → $3.3\times$ score. LLM model size is the primary performance lever.



Key Findings

1. Off-policy replay is decisive. DQN's 100K replay buffer (S&B §8.2) recycles rare high-tile transitions that on-policy methods see only once. PPO score = 962; DQN = 7,744 on identical CNN. Replay gap = $8\times$.

2. 120 params beats 330K+ at sample efficiency. Semi-gradient SARSA with hand-crafted features (monotonicity, snake, corner) outperforms all deep on-policy agents. Validates S&B §9.5: strong inductive bias beats raw capacity when data is scarce.

3. More training actively hurts on-policy agents. PPO -12% and QR-DQN -9% from 1M→5M. PPO develops degenerate tile-sliding loops; QR-DQN collapses under distributional overfitting. On-policy methods *unlearn* good behavior.

4. LLM scale drives RLVR performance. $3\times$ parameters (0.5B → 1.5B) → $3.3\times$ avg score. GRPO uniquely produces *interpretable* per-move rationale. At larger scale (3B+), GRPO is the strongest candidate to close the gap.

Future Work

- **Replay buffer ablation:** vary DQN buffer (10K/50K/100K/500K) to quantify exactly how much replay drives the $8\times$ PPO gap
- **Prioritized Experience Replay (PER):** over-sample rare high-tile transitions; directly motivated by Key Finding 1
- **Spatial board encoding for GRPO:** replace list-of-lists with labeled ASCII grid to test if spatial failures decrease
- **Extended GRPO training:** 300 → 1K+ prompts on Colab A100 (free tier); directly tests whether the scaling law continues
- **Multi-seed eval** (n=5) for all classical agents to establish statistical significance of SARSA vs. deep on-policy gap

References: [1] Sutton & Barto, *RL: An Introduction*, 2e, MIT Press, 2018. [2] Mnih et al., Nature 518, 2015. [3] Schulman et al., arXiv:1707.06347, 2017. [4] Guo et al., DeepSeek-R1, 2025. [5] Shao et al., DeepSeekMath, 2024. [6] Christodoulou, Discrete SAC, arXiv:1910.07207, 2019.